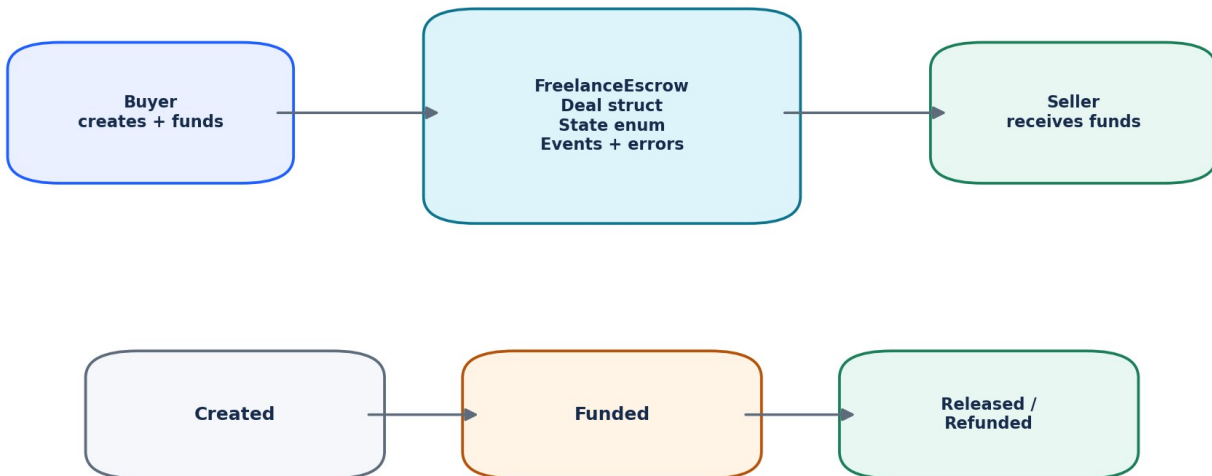


DAY 6

Solidity Fundamentals II

Constructors, Modifiers, Errors, Events, Structs, Enums, Inheritance, Interfaces, Libraries & Ether Entry Points

60-MINUTE OUTCOME Students progress from isolated Solidity syntax to contract design patterns: initialization, authorization, failure handling, state machines, logs, code reuse, and controlled native-coin reception.



Lecture	Duration	Portfolio Outcome
Day 6	60 minutes	A beginner FreelanceEscrow contract plus design/debugging exercises
Level	Beginner	Assumes Day 5: variables, functions, arrays, mappings, data locations
Career focus	LinkedIn + Upwork	Understand and modify real Solidity contracts that use common production patterns

SAFETY NOTE The escrow code is educational. Real contracts holding value need comprehensive tests, threat modeling, access-control review, and security assessment before production use.

1. Learning Objectives & Minute-by-Minute Schedule

By the end of Day 6, students should be able to read a more realistic contract and explain initialization, access checks, state transitions, failure paths, event logs, inheritance relationships, and Ether entry points.

- Use constructors and immutable values for deployment-time configuration.
- Write modifiers for reusable preconditions and explain the underscore placeholder.
- Compare require, revert with custom errors, and assert.
- Define and emit events and explain indexed parameters at a beginner level.
- Model grouped data with structs and state machines with enums.
- Use inheritance, abstract contracts, interfaces, and libraries for code organization.
- Explain receive() and fallback() and when each is selected.
- Build a small escrow contract that combines these ideas.

Minutes	Topic	Instructor Outcome
0-6	Day 5 review + constructor	Connect persistent state to deployment-time initialization.
6-14	Modifiers + access checks	Students understand reusable function gates.
14-22	require / revert / errors / assert	Students can classify common failure mechanisms.
22-29	Events and logs	Students know how contracts communicate activity to off-chain apps.
29-37	Structs and enums	Students model records and lifecycle states.
37-47	Inheritance, interfaces, libraries	Students understand three different reuse/composition tools.
47-52	receive() and fallback()	Students understand native-coin and unmatched-call entry points.
52-58	Hands-on FreelanceEscrow	Combine the concepts in one coherent contract.
58-60	Quiz + freelancer task + homework	Check understanding and assign practice.

TEACHING METHOD For each construct, ask four questions: When does it execute? What state can it read/write? What happens if it fails? How would a frontend or client observe the result?

2. Quick Day 5 Review | 0-2 min

Before adding advanced syntax, verify that students still understand the Day 5 foundation.

Question	Expected Answer
Where does a state variable live?	In persistent contract storage/state.
What does external mean?	The function is intended to be called from outside the contract through the external call interface.
What does view promise?	The function does not modify blockchain state.
storage vs memory?	storage refers to persistent state; memory is temporary mutable data for one call.
What does msg.sender represent?	The immediate caller of the current external/message call.

CHECKPOINT If a student cannot distinguish storage from memory or public from private, revisit Day 5 before continuing. Day 6 builds heavily on those concepts.

3. Constructors and Deployment-Time Initialization | 2-6 min

A constructor is a special function that executes once when a contract is created. It is commonly used to initialize owner/admin addresses, immutable configuration, external contract addresses, and starting state.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract ProjectConfig {
    address public immutable client;
    address public immutable developer;

    constructor(address developer_) {
        client = msg.sender;
        developer = developer_;
    }
}
```

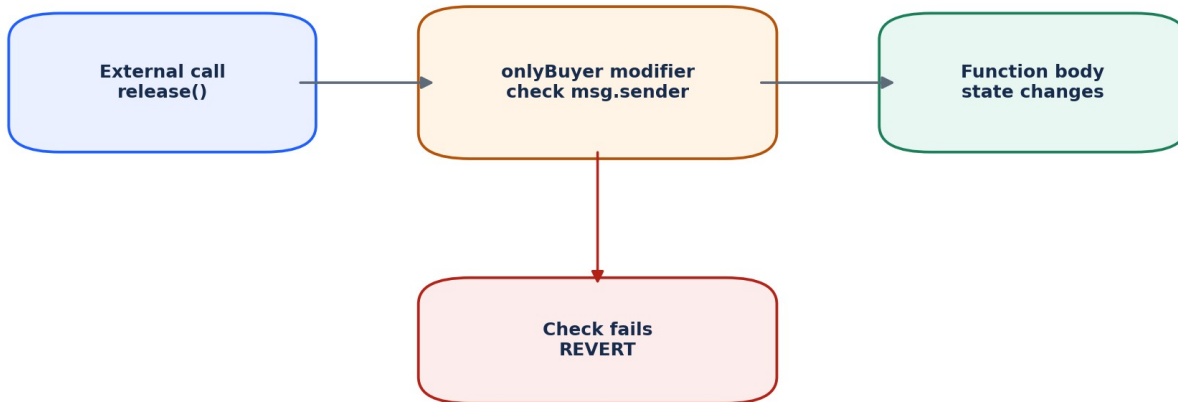
Important constructor facts

- A contract has at most one constructor.
- The constructor runs during contract creation, not on later normal calls.
- constructor arguments are encoded into the deployment transaction.
- msg.sender in the constructor is the account or contract that creates the contract.
- immutable variables can be assigned during construction and then cannot be reassigned by ordinary logic.

FREELANCER BUG A surprisingly wrong owner often starts at deployment: the constructor was given the wrong address, a factory deployed the contract, or the deployer account was not the account the client expected.

4. Function Modifiers | 6-14 min

A modifier wraps reusable logic around a function body. Modifiers are often used for access control, lifecycle/state checks, pause checks, deadlines, or other preconditions.



The underscore (`_`) marks where the modified function body executes.

```

contract SimpleAdmin {
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "Not owner");
        _;
    }

    function adminAction() external onlyOwner {
        // This body runs where the underscore ( _ ) appears.
    }
}
  
```

What does the underscore mean?

The underscore statement is replaced conceptually by the body of the modified function. Code before the underscore runs before the function body; code after the underscore runs after the function body.

```

modifier example() {
    // BEFORE function body
    _;
    // AFTER function body
}
  
```

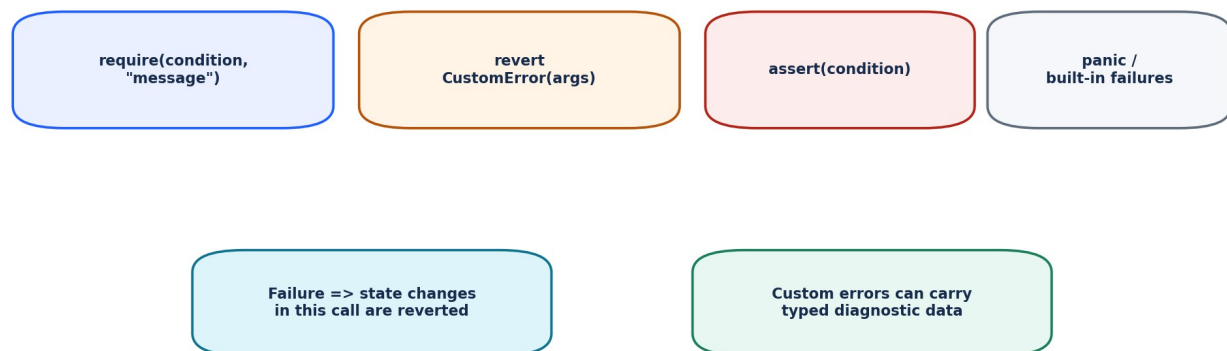
Common modifier patterns

Pattern	Typical Check	Example Use
onlyOwner	msg.sender == owner	Admin configuration
onlyBuyer	msg.sender == buyer	Escrow approval
whenNotPaused	paused == false	Emergency pause systems
inState	state == expected	State-machine lifecycle
beforeDeadline	block.timestamp < deadline	Timed sales/claims

DESIGN RULE Modifiers should make authorization and preconditions clearer. Do not hide complex business logic inside a modifier just because Solidity allows code before or after the underscore.

5. require, revert, Custom Errors & assert | 14-22 min

Smart contracts need explicit failure paths. A failed/reverted call rolls back state changes made by that call, although transaction gas can still be consumed.



require

```
require(msg.value > 0, "Amount must be greater than zero");
```

Use require for readable precondition checks, especially in beginner code. In mature code, custom errors are often preferred for structured, gas-efficient reverts.

Custom errors + revert

```
error NotBuyer(address caller);
error InvalidState(uint8 current);

if (msg.sender != buyer) {
    revert NotBuyer(msg.sender);
}
```

assert

```
assert(address(this).balance >= deal.amount);
```

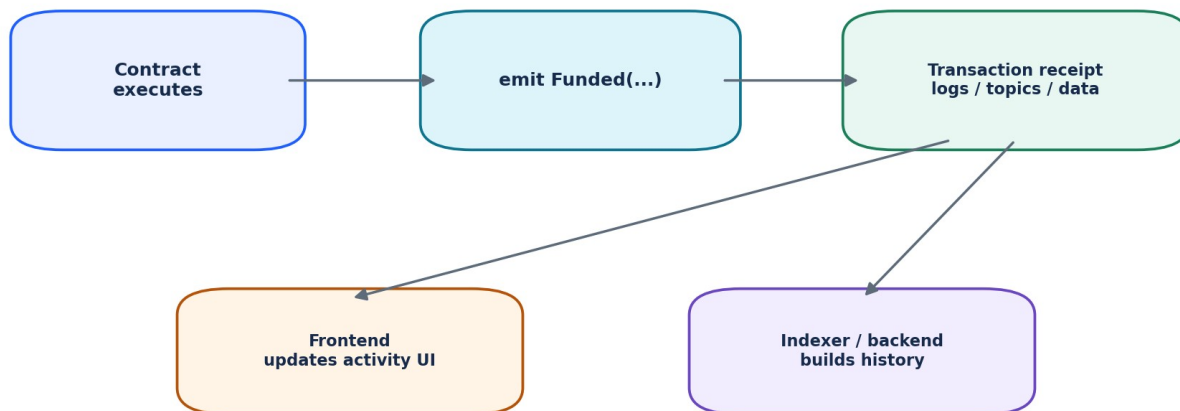
assert is intended for invariants and conditions that should never be false if the contract logic is correct. It is not the normal tool for validating user input.

Mechanism	Good Beginner Use	Do Not Confuse With
require(condition, message)	Input/precondition validation	An authorization system by itself; the condition must actually be correct.
revert CustomError(...)	Typed, explicit failure path	Returning false. A revert aborts the call.
assert(condition)	Internal invariant / impossible state check	User validation such as "amount > 0".

CURRENT SOLIDITY NOTE Official Solidity documentation describes custom errors as descriptive, typed failure data usable with revert statements. They can encode additional diagnostic values and are cheaper than long revert strings.

6. Events and Logs | 22-29 min

Events let a contract write structured log entries into a transaction receipt. Frontends, indexers, analytics systems, and monitoring services can watch those logs without treating them as contract storage.



Events are durable log records for off-chain consumers; they are not contract storage.

```

event Funded(address indexed buyer, uint256 amount);
event Released(address indexed seller, uint256 amount);

function fund() external payable {
    // ... validation and state update ...
    emit Funded(msg.sender, msg.value);
}
  
```

Why indexed?

Indexed event parameters are placed into log topics, which makes them useful for filtering/searching. A common pattern is to index addresses or IDs that applications frequently query.

Element	Meaning
event declaration	Defines the log schema.
emit	Writes an event log during successful execution.
indexed parameter	Can be represented in log topics for filtering.

non-indexed parameter	Encoded in the event data area.
transaction receipt	Contains status, gas information, and emitted logs.

FREELANCER SCENARIO Client says: "The contract succeeded but our dashboard never updated." Check the transaction receipt and event logs, then check whether the frontend is listening to the correct contract address, network, event signature, and block range.

7. Structs - Group Related Data | 29-33 min

A struct is a custom data type that groups related fields. It is useful when one entity has several properties that should be handled together.

```
struct Deal {
    uint256 amount;
    uint64 createdAt;
    address seller;
    bool funded;
}

Deal public deal;
```

Struct inside a mapping

```
mapping(uint256 => Deal) public deals;

function createDeal(uint256 id, address seller) external {
    deals[id] = Deal({
        amount: 0,
        createdAt: uint64(block.timestamp),
        seller: seller,
        funded: false
    });
}
```

Use a struct when...	Example
One logical object has multiple fields	User profile: name, wallet, score, active flag
You need mapping ID => record	orderId => Order
You need an array of records	Proposal[] proposals
A function returns grouped information	getDeal() returns Deal memory

STORAGE NOTE A struct stored in contract state consumes storage for its fields. Grouping fields improves code clarity but does not make storage free.

8. Enums - Model Finite States | 33-37 min

An enum defines a fixed set of named choices. It is ideal for lifecycle/state-machine logic because code becomes clearer than using unexplained integer values.

```
enum State {
    Created,
    Funded,
    Released,
    Refunded
}
```

```
State public state = State.Created;
```

State	Meaning	Allowed Next Example
Created	Escrow exists but has no funds	Funded
Funded	Buyer deposited funds	Released or Refunded
Released	Funds sent to seller	Terminal
Refunded	Funds returned to buyer	Terminal

CLIENT VALUE Enums are especially useful when a client asks: "Why is this transaction invalid now?" The answer often comes from the contract being in a state where that action is not allowed.

9. Inheritance | 37-41 min

Inheritance lets one contract reuse and extend functions/state from a base contract. Solidity supports single and multiple inheritance, but beginners should first master simple linear cases.

```
contract OwnableLite {
    address public owner;

    constructor() { owner = msg.sender; }

    modifier onlyOwner() {
        require(msg.sender == owner, "Not owner");
        _;
    }
}

contract ProjectRegistry is OwnableLite {
    uint256 public projectCount;

    function addProject() external onlyOwner {
        projectCount++;
    }
}
```

virtual and override

```
contract Base {
    function label() public pure virtual returns (string memory) {
        return "Base";
    }
}

contract Child is Base {
    function label() public pure override returns (string memory) {
        return "Child";
    }
}
```

BEGINNER RULE Use inheritance when the child truly is a specialized version of the base behavior. Do not create deep inheritance trees merely to avoid copying a few lines.

10. Interfaces | 41-44 min

An interface describes functions/events a contract exposes without providing the full implementation. It is a contract-to-contract/API boundary: “I know this address supports these callable functions.”

```
interface IERC20Like {
    function balanceOf(address account) external view returns (uint256);
    function transfer(address to, uint256 amount) external returns (bool);
}

contract BalanceReader {
    function read(address token, address user) external view returns (uint256) {
        return IERC20Like(token).balanceOf(user);
    }
}
```

IMPORTANT An interface does not verify that an arbitrary address truly implements the expected behavior. Calls can revert or return unexpected data. Production integrations still require validation and testing.

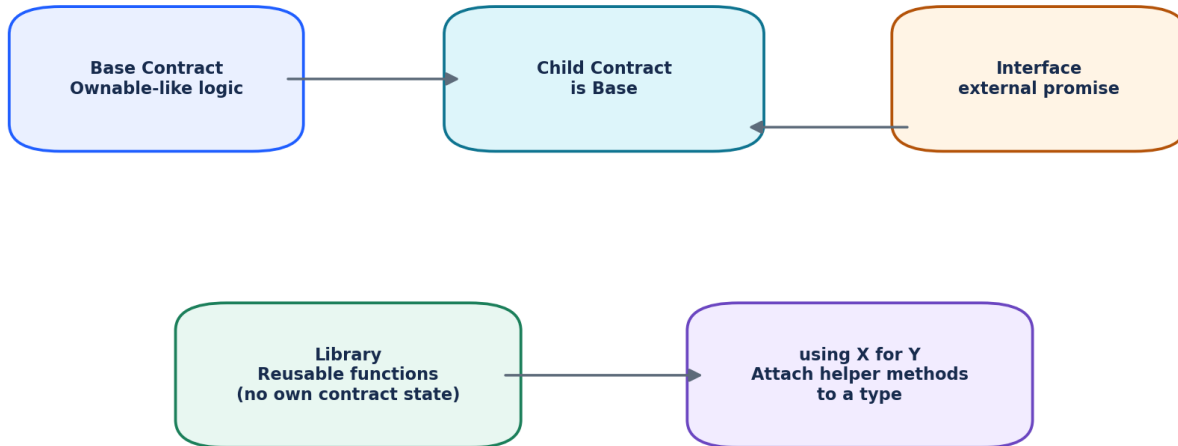
11. Libraries | 44-47 min

Libraries package reusable functions. They are especially useful for common operations that do not need independent mutable contract state. Solidity also supports “using Library for Type” syntax.

```
library MathTools {
    function percent(uint256 amount, uint256 bps)
        internal pure returns (uint256)
    {
        return amount * bps / 10_000;
    }
}

contract FeePreview {
    using MathTools for uint256;

    function fee(uint256 amount, uint256 bps)
        external pure returns (uint256)
    {
        return amount.percent(bps);
    }
}
```



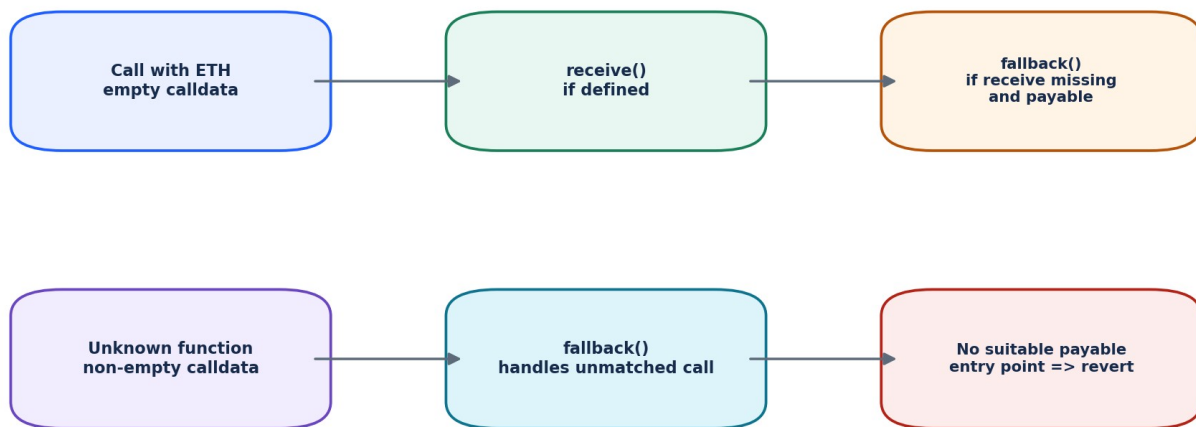
Inheritance reuses implementation; interfaces define callable shape; libraries package reusable logic.

Construct	Primary Purpose	Mental Model
Inheritance	Reuse/extend implementation	Child IS-A specialized base contract
Interface	Describe external callable shape	This address SHOULD expose these functions
Library	Package reusable functions	Helper logic shared by contracts

OFFICIAL LANGUAGE RULE Solidity interfaces and libraries have stricter grammar than normal contracts. For example, interfaces cannot contain state variables, and libraries do not define constructors/receive/fallback like ordinary contracts.

12. receive() and fallback() | 47-52 min

Contracts can receive calls that do not match an ordinary named function. Solidity provides two special external entry points: receive and fallback.



receive()

```

receive() external payable {
    emit NativePayment(msg.sender, msg.value);
}

```

receive() is selected for a plain native-coin transfer with empty calldata when the contract defines a receive function.

fallback()

```

fallback() external payable {
    revert("Unknown call");
}

```

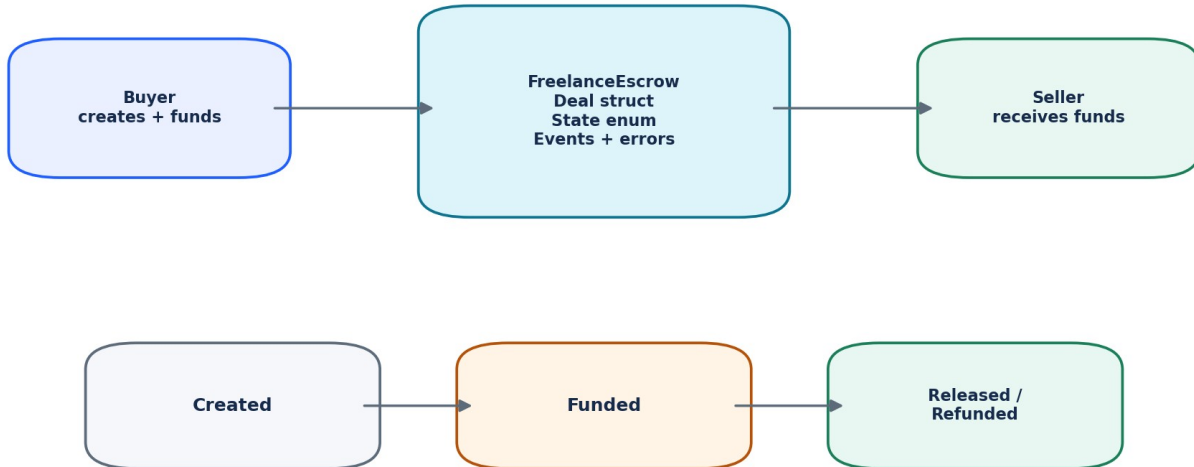
fallback() is selected when calldata does not match another function. If no receive function exists, a payable fallback can also receive a plain Ether transfer.

Situation	Likely Entry Point
ETH sent with empty calldata and receive exists	receive()
Unknown function selector / non-empty unmatched calldata	fallback()
Plain ETH, no receive, payable fallback exists	fallback()
No suitable payable receive/fallback	Transaction reverts

DESIGN CHOICE Do not add payable receive/fallback just because you can. Decide whether the contract should accept direct native-coin transfers. Many application contracts intentionally reject unexpected ETH.

13. Hands-On Project: FreelanceEscrow | 52-58 min

Goal: combine Day 6 constructs in a contract that models a simple buyer-seller escrow. The buyer funds the contract and can release payment; the seller can approve a refund. This is intentionally simple so students can read every line.



```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract FreelanceEscrow {
    enum State { Created, Funded, Released, Refunded }

    struct Deal {
        uint256 amount;
        uint64 createdAt;
        State state;
    }

    address public immutable buyer;
    address public immutable seller;
    Deal public deal;

    error NotBuyer(address caller);
    error NotSeller(address caller);
    error InvalidState(State expected, State actual);
    error InvalidAmount();
    error TransferFailed();
    error DirectPaymentNotAllowed();

    event Funded(address indexed buyer, uint256 amount);
    event Released(address indexed seller, uint256 amount);
    event Refunded(address indexed buyer, uint256 amount);

    constructor(address seller_) {
        require(seller_ != address(0), "Seller is zero address");
        buyer = msg.sender;
        seller = seller_;
        deal = Deal({
            amount: 0,
            createdAt: uint64(block.timestamp),
            state: State.Created
        });
    }
}
```

```

    });
}

modifier onlyBuyer() {
    if (msg.sender != buyer) revert NotBuyer(msg.sender);
    _;
}

modifier onlySeller() {
    if (msg.sender != seller) revert NotSeller(msg.sender);
    _;
}

modifier inState(State expected) {
    if (deal.state != expected) {
        revert InvalidState(expected, deal.state);
    }
    _;
}

function fund() external payable onlyBuyer inState(State.Created) {
    if (msg.value == 0) revert InvalidAmount();
    deal.amount = msg.value;
    deal.state = State.Funded;
    emit Funded(msg.sender, msg.value);
}

function release() external onlyBuyer inState(State.Funded) {
    uint256 amount = deal.amount;
    deal.amount = 0;
    deal.state = State.Released;

    (bool ok, ) = payable(seller).call{value: amount}("");
    if (!ok) revert TransferFailed();
    emit Released(seller, amount);
}

function refund() external onlySeller inState(State.Funded) {
    uint256 amount = deal.amount;
    deal.amount = 0;
    deal.state = State.Refunded;

    (bool ok, ) = payable(buyer).call{value: amount}("");
    if (!ok) revert TransferFailed();
    emit Refunded(buyer, amount);
}

receive() external payable {
    revert DirectPaymentNotAllowed();
}

fallback() external payable {
    revert DirectPaymentNotAllowed();
}
}

```

14. Walk Through the Escrow Line by Line

Construct	Where It Appears	What It Teaches
enum State	Created/Funded/Released/Refunded	Finite lifecycle and readable state machine
struct Deal	amount + createdAt + state	Grouped business record
immutable	buyer + seller	Deployment-time configuration
constructor	seller_ + buyer setup	One-time initialization
custom errors	NotBuyer, InvalidState, etc.	Typed failure information
events	Funded/Released/Refunded	Off-chain activity log
modifiers	onlyBuyer/onlySeller/inState	Reusable authorization/state gates
payable	fund()	Function can receive native coin
call{value: ...}	release/refund	Native-coin transfer with success check
receive/fallback	both revert	Reject unexpected direct payments/calls

Why update state before the external payment call?

The contract sets amount to zero and advances the lifecycle before calling the recipient. This follows the Checks-Effects-Interactions idea that Day 10 will study in security depth. If the call fails and the transaction reverts, those state updates also roll back.

SECURITY PREVIEW External calls are one of the most important smart-contract risk boundaries. Day 6 introduces the safe mental model; Day 10 will cover reentrancy and adversarial external-call behavior in detail.

Hands-on Remix sequence

1. Compile with a compatible Solidity 0.8.x compiler.
2. Deploy FreelanceEscrow from Account A, passing Account B as seller.
3. Verify buyer == Account A and seller == Account B.
4. From Account A, call fund() with a small test value in Remix VM.
5. Read deal() and confirm amount/state changed.
6. Try release() from the seller account; observe NotBuyer.
7. Switch back to buyer and call release().
8. Inspect the Released event and final deal state.
9. Deploy a fresh instance, fund it, then switch to seller and test refund().

15. Debugging Exercises

Exercise A - “Not owner” / “Not buyer”

A client says a function always reverts. The explorer shows the caller is a different address than the configured buyer.

- Check the constructor arguments and deployment transaction.
- Read the buyer/owner variable on-chain.
- Compare it with the transaction from address.
- Explain that wallet UI account switching changes msg.sender.

Exercise B - Wrong state

fund() succeeded. The user tries fund() a second time and receives InvalidState.

- Read deal.state.
- Map the enum numeric value to its named state.
- Explain the state machine: Created -> Funded, so a second fund is intentionally rejected.

Exercise C - Frontend missed activity

Transaction succeeded, but dashboard history is empty.

- Check receipt logs for Funded/Released/Refunded.
- Check contract address and chain ID in the frontend.
- Check ABI event definitions and indexed-filter values.
- Check the indexer/listener start block and RPC provider.

Exercise D - Unexpected ETH transfer reverts

A client sends ETH directly to the escrow address instead of calling fund(). The transfer reverts.

Correct explanation: this contract intentionally rejects direct payments in receive/fallback. The designed entry point is fund(), where state and amount are updated together.

FREELANCER HABIT When debugging a revert, identify the exact function/entry point, msg.sender, msg.value, current state, and decoded error before changing code.

16. Upwork / LinkedIn Client Scenarios

Client Request	What You Should Think About	Day 6 Skills
"Add an admin-only function."	Who is admin? How initialized? Can ownership change?	constructor + modifier + custom error
"We need order status on-chain."	Finite states and valid transitions	enum + modifier + event
"Store project information."	Which fields belong together? How retrieved?	struct + mapping/array from Day 5
"Integrate an ERC-20 contract."	What callable API do we need?	interface
"Share fee math across contracts."	Reusable stateless helper logic	library
"Listen when payment happens."	What event schema and indexed fields?	events/logs
"Why did ETH transfer revert?"	receive/fallback/payable behavior	special entry points
"Create a base contract for multiple products."	Is inheritance really appropriate?	inheritance + override

How to explain your implementation to a client

A professional response should describe behavior, not just syntax. Example: "The buyer and seller are fixed at deployment, payment can only enter through fund(), the contract records a Funded event, and release/refund are gated by both caller role and current escrow state."

PORTFOLIO IDEA Publish the escrow as an educational repository with an architecture diagram, state-transition table, test cases, deployment instructions, and explicit "not audited / educational" disclaimer.

17. Common Beginner Mistakes

Mistake	Why It Is Wrong / Risky
Using private to protect secrets	private only restricts Solidity-level access; blockchain state is observable.
Modifier checks the wrong address	Access control is only as correct as its condition.
Using assert for user input	assert is for invariants/internal correctness, not normal caller validation.
Expecting an event to change state	Events are logs. Contract logic must update storage separately.
Using enum integer without mapping name	Explorers may display numeric values; developers must know the enum order.
Assuming interface proves behavior	It only describes the call shape used by your code.
Making fallback payable by default	It may unintentionally accept native coin.
Sending value before updating state	External calls can introduce reentrancy/security concerns.

18. Interview Questions & Strong Beginner Answers

Question	Strong Answer
1. What is a constructor?	A special function executed once when the contract is created, usually for initial configuration.
2. What is a modifier?	Reusable logic that wraps a function body, often used for access or state preconditions.
3. What does the underscore in a modifier mean?	It marks where the modified function body executes.
4. require vs revert?	Both can abort execution; require is a convenient condition check, while revert explicitly aborts and can use custom errors.
5. When should assert be used?	For invariants/internal conditions that should never fail if the program is correct.
6. What is an event?	A structured log emitted during execution and stored in the transaction receipt for off-chain consumers.
7. What is a struct?	A custom type grouping multiple fields into one record.
8. What is an enum?	A fixed set of named values useful for finite states.
9. Inheritance vs interface?	Inheritance reuses/extends implementation; an interface describes callable functions without implementing the contract logic.
10. What is a library?	A reusable collection of functions, commonly for helper logic.
11. receive vs fallback?	receive handles empty-call data native transfers when defined; fallback handles unmatched calls and can receive value if payable.
12. Why emit an event after a state change?	It gives off-chain systems a durable activity record that corresponds to successful contract execution.

19. Day 6 Mini Quiz

Answer without looking at the notes. Suggested time: 2-3 minutes in class; finish the rest as homework if necessary.

1. When does a constructor run?
2. What does `_;` do inside a modifier?
3. What happens to state writes when a transaction reverts?
4. Give one reason to prefer a custom error over a long revert string.
5. Are event logs the same as contract storage?
6. What does indexed mean in an event parameter?
7. When is a struct more appropriate than several unrelated variables?
8. Why is an enum useful for an escrow lifecycle?
9. What is the conceptual difference between inheritance and an interface?
10. What does a library provide?
11. Which special function handles an unmatched function call?
12. A plain ETH transfer with empty calldata arrives. Which function is preferred if it exists?
13. Why might a contract deliberately reject direct ETH transfers?
14. Why should external calls be treated as a security boundary?

Answer Key

1) During contract creation. 2) It inserts/runs the modified function body at that point. 3) They roll back. 4) Typed data and/or lower gas/bytecode cost than long strings. 5) No, events are logs. 6) The value is placed in a topic for filtering (subject to ABI rules). 7) When fields belong to one logical record. 8) It limits state to named lifecycle choices. 9) Inheritance reuses implementation; interface describes callable shape. 10) Reusable helper functions. 11) `fallback()`. 12) `receive()`. 13) To force payments through business logic that records state/validates conditions. 14) Called code can fail or behave adversarially, including reentrancy patterns.

20. Homework - 45 to 90 Minutes

Task A - Extend the escrow

- Add a string projectId or bytes32 jobId to the Deal struct.
- Emit the job ID in Funded/Released/Refunded events.
- Add a view function named currentState() that returns the enum.
- Add a modifier that rejects the zero seller address during construction OR explain why constructor require is simpler here.

Task B - Write three failure tests manually in Remix

- Seller attempts release() -> should revert NotBuyer.
- Buyer calls release() before fund() -> should revert InvalidState.
- User sends native coin directly to the contract -> should revert DirectPaymentNotAllowed.

Task C - Interface exercise

Write a minimal interface for a contract with functions owner() view returns (address), paused() view returns (bool), and pause(). Do not implement the functions.

Task D - Freelancer explanation

Write a 150-word client update explaining how the escrow prevents unauthorized release, how lifecycle state works, and how events allow the frontend to build transaction history.

PORTFOLIO CHECKPOINT Your Day 6 repository should contain: source contract, README, state diagram, screenshots of successful fund/release/refund in Remix, and screenshots of at least two intentional reverts.

21. Day 6 Cheat Sheet

Keyword / Pattern	Remember
constructor	Runs once at deployment; initialize configuration.
immutable	Assigned at construction, then not reassigned.
modifier	Reusable wrapper/check around function body.
_;	Where the modified function body executes.
require	Convenient condition/precondition check.
error + revert	Typed failure with optional data.
assert	Internal invariant / correctness condition.
event + emit	Write structured transaction logs.
indexed	Event field usable as a log topic/filter.
struct	Group related fields into a record.
enum	Fixed set of named lifecycle values.
is	Inheritance declaration.
virtual / override	Allow and implement function overriding.
interface	Describe an external contract API.
library	Reusable helper functions.
receive()	Empty-calldata native transfer entry point.
fallback()	Unmatched call entry point.
payable	Function/address context can receive/send native coin as allowed.

22. Instructor Notes & Suggested Whiteboard Flow

If teaching live, use the following whiteboard sequence. Each line should be drawn while students explain what changes.

```

DEPLOY
  constructor -> buyer/seller -> State.Created

FUND
  modifier checks -> msg.value -> Deal.amount -> State.Funded -> emit Funded

RELEASE
  onlyBuyer + inState(Funded)
  -> amount = deal.amount
  -> amount=0, State.Released
  -> external payment
  -> emit Released

OR REFUND
  onlySeller + inState(Funded)
  -> State.Refunded -> buyer paid -> emit Refunded

```

Three questions to repeat during the lecture

- Who is allowed to call this?
- What state must exist before the call?
- What state/logs exist after a successful call?

Optional 5-minute extension if class moves quickly

Ask students how they would add a third-party arbiter. Do not code it immediately. First define the actors, allowed state transitions, and who can call refund/release. This teaches requirements analysis before implementation.

23. Primary Technical References

Use primary documentation when students want to go deeper or when compiler behavior differs from an old tutorial:

- Solidity documentation - Structure of a Contract: errors, events, functions, modifiers, structs and enums.
- Solidity documentation - Contracts: inheritance, abstract contracts, interfaces, libraries, receive and fallback functions.
- Solidity documentation - Expressions and Control Structures: revert behavior and error handling.
- Solidity documentation - ABI Specification: external call and event encoding concepts.
- OpenZeppelin Contracts 5.x documentation - Access Control: Ownable and role-based access-control patterns.

VERSION PRACTICE Examples use pragma ^0.8.24 for broad modern 0.8.x compatibility. In real client work, compile with the project-pinned compiler version and read that version's documentation/release notes before changing behavior.

24. Preparation for Day 7

Day 7 moves from language fundamentals into a major client deliverable: ERC-20 token development.

- ERC-20 standard functions and events
- totalSupply, balanceOf, transfer, approve, allowance, transferFrom
- OpenZeppelin ERC20 implementation
- minting and burning
- Ownable vs AccessControl roles
- supply caps and pause patterns
- deployment, verification, and basic tests